

# Web-to-Warehouse Product Intelligence Pipeline

A compliance-first, orchestrated and testable ETL platform that turns permitted public web sources into a dimensional warehouse with full price history, duplicate resolution, catalog reconciliation and a measured data-quality posture.

DEPI · Data Engineering

MIT licensed

Private repository

# The problem

Manual collection is slow, inconsistent, and destroys the history that makes pricing decisions possible.

## Slow

- Analyst hours spent copying prices into spreadsheets
- A refresh cycle measured in days, not minutes

## Inconsistent

- Same product named three ways
- Currencies and formats never normalised

## Amnesiac

- No history, so no trend
- No audit trail, so no defensible claim

■ **The question nobody can answer:** what did this product cost last March, and on whose authority?

# Objective and scope

Collect permitted product data, standardise it, resolve duplicates, and load it into an analytical database that answers business questions.

## In scope

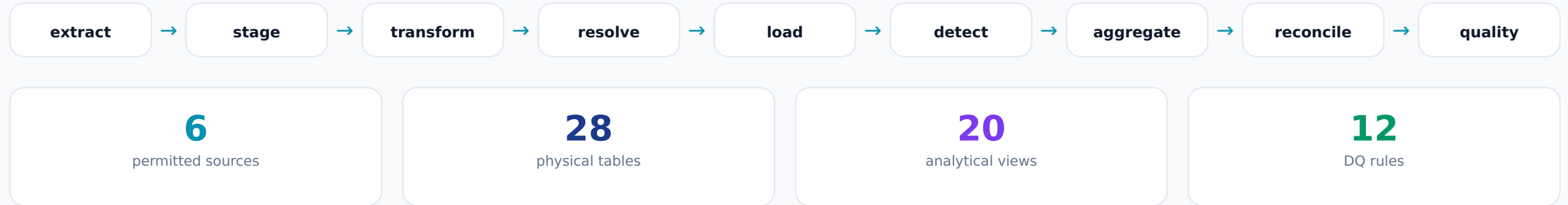
- Scrape permitted structured product information
- Respect robots.txt, rate limits and website terms
- Clean names and categories; normalise currencies
- Detect duplicates and compare against the internal catalogue
- Load into PostgreSQL / MySQL with historical snapshots
- Identify price, new, removed and category changes with SQL
- Orchestrate with Airflow; expose the result over REST

## Deliberately out of scope

- Bypassing any site's protections
- Personal or behavioural data
- Real-time streaming ingestion
- Training a machine-learning model
- Anything not evidenced by a test

# The pipeline, end to end

Nine stages, orchestrated by Airflow, each reporting progress and each writing evidence of what it did.



Every stage publishes progress to a leased job queue, so a slow run is visible rather than silent.

# Compliance is evidence, not a claim

The ingestion layer is where a scraper earns the right to run.

## Enforced in the fetch layer

- robots.txt parsed and its verdict recorded per request
- Per-source rate limits and crawl-delay honoured
- Conditional requests and an on-disk cache
- An identifying User-Agent with a contact address

## Recorded as proof

- Every request logged with URL, status and elapsed time
- The robots verdict stored alongside each request
- Cache hits distinguished from live fetches
- One audit row per pipeline run, queryable by screen

■ A reviewer can reconstruct **what was fetched, when, and whether it was permitted** without reading the code.

# Warehouse and schema ownership

A Kimball star schema, owned by a migration rather than by convention.

## Dimensions

- dim\_product
- dim\_category
- dim\_source
- dim\_date
- dim\_currency

## Facts

- fact\_price\_snapshot &mdash; the historical price table
- fact\_catalog\_snapshot &mdash; our own list prices
- chg\_price\_change &mdash; detected movements
- chg\_product\_event &mdash; lifecycle

## Aggregates

- agg\_category\_daily
- agg\_brand\_monthly
- Materialised during the run

Alembic creates all 28 tables and applies all 20 views; `alembic check` fails the build when the models and the migration disagree.

# Beyond history: what the data says next

History is already in the fact table. These three things are not.

## Forecasting

- Damped Holt-Winters with two-sided bands
- Accuracy measured on a held-out tail, never on the fit

## Anomaly detection

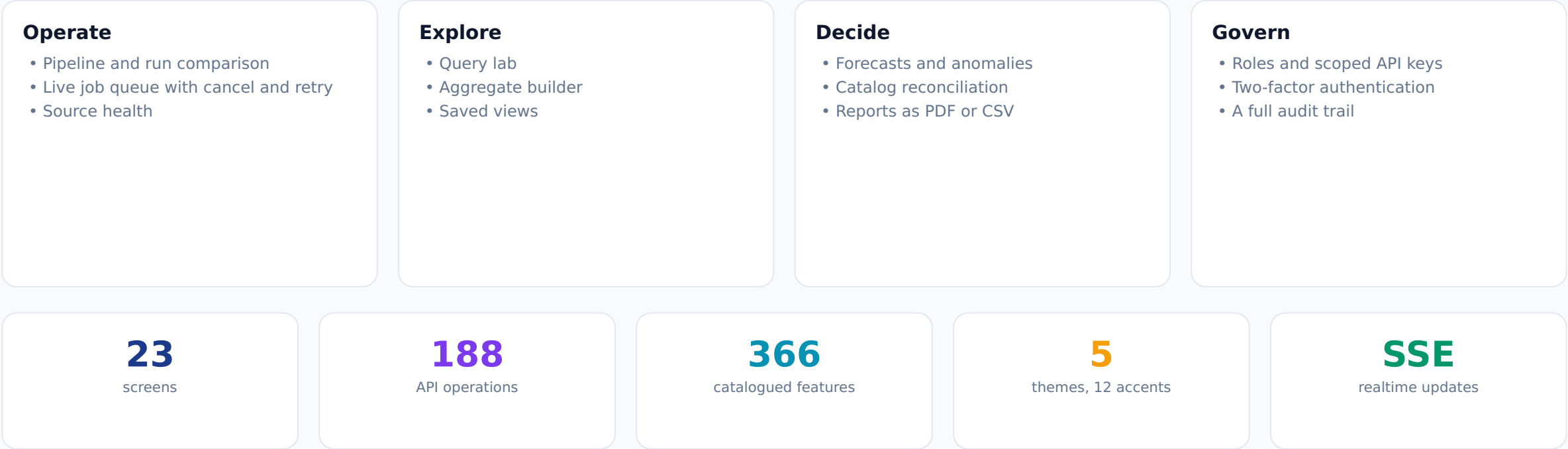
- MAD,  $3\sigma$ ; and IQR fences run together
- Two of three must agree, so an outlier cannot pass its own test

## Pricing advice

- Category elasticity
- Raise, cut, hold or negotiate — with the reason stated

Where the history is too short, the API returns a flat series with wide bands rather than a confident-looking curve.

# What the user actually gets





# Verification

What has to pass before this counts as working.

| Gate                                                  | Result          |
|-------------------------------------------------------|-----------------|
| pytest (unit + integration)                           | 449 passed      |
| API smoke, against the running stack                  | 104 passed      |
| Ruff (app, tests, scripts, DAGs, migrations)          | clean           |
| mypy                                                  | clean, 83 files |
| TypeScript, ESLint at zero warnings, production build | clean           |
| Render smoke test, every screen                       | 23 screens      |
| alembic check                                         | no drift        |
| Mermaid diagrams rendered                             | 80 of 80        |
| Documentation links resolved                          | 1216            |

Structural figures are measured by `scripts/project_stats.py` and pinned by `scripts/check_stats.py`, which fails when a documented number stops matching the code.

# Documentation as a deliverable



# What made it hard, and what was done about it

## Duplicate resolution

- The same product appears under three names on three sites
- Blocking keys to bound the comparison set
- A similarity threshold, with the strategy recorded per match

## Rebuilding a schema in place

- create\_all cannot express a change, a rollback or a history
- Alembic adopted; Airflow metadata moved to its own database

## Realtime across workers

- An in-memory broker only sees its own process
- Events now also polled from a persisted log

The one that was **not** caught by any static check: an external store returning a new snapshot object per call, which put every screen into an infinite render loop. It is now pinned by a test.

# Delivered

A working pipeline, a dimensional warehouse with history, a measured quality posture, a REST API, a dashboard, and documentation that a reviewer can verify rather than take on trust.

Ingestion module

Database schema

Historical price table

ETL pipeline

Airflow DAG

Data-quality checks

SQL analysis

Visualisation

REST API

Documentation